

FY-26: Career Preview Program project Review (CPP-3)

College /University Name: JECRC Foundation

CXL Enablement with QEMU-Based Emulation

College Name: **JECRC Foundation**

 **Github:** github.com/CXL-Enablement-with-QEMU-Based-Emulation.git

HPE Mentor

Mohan Parthasarathy
Tharunkumar Radharapu
Renganathan MeenakshiSundaram
Pranjal Goswami

Faculty Mentor

Ruchika Solanki

Project Participants

Aadhar Bindal
Madhur Kedia
Ronak Khandelwal
Virendra Singh Rathore
Vishwas Saini



JECRC Foundation

Need for CXL



EXPONENTIAL COMPUTE GROWTH

CPU cores and performance are scaling faster than memory bandwidth and capacity.



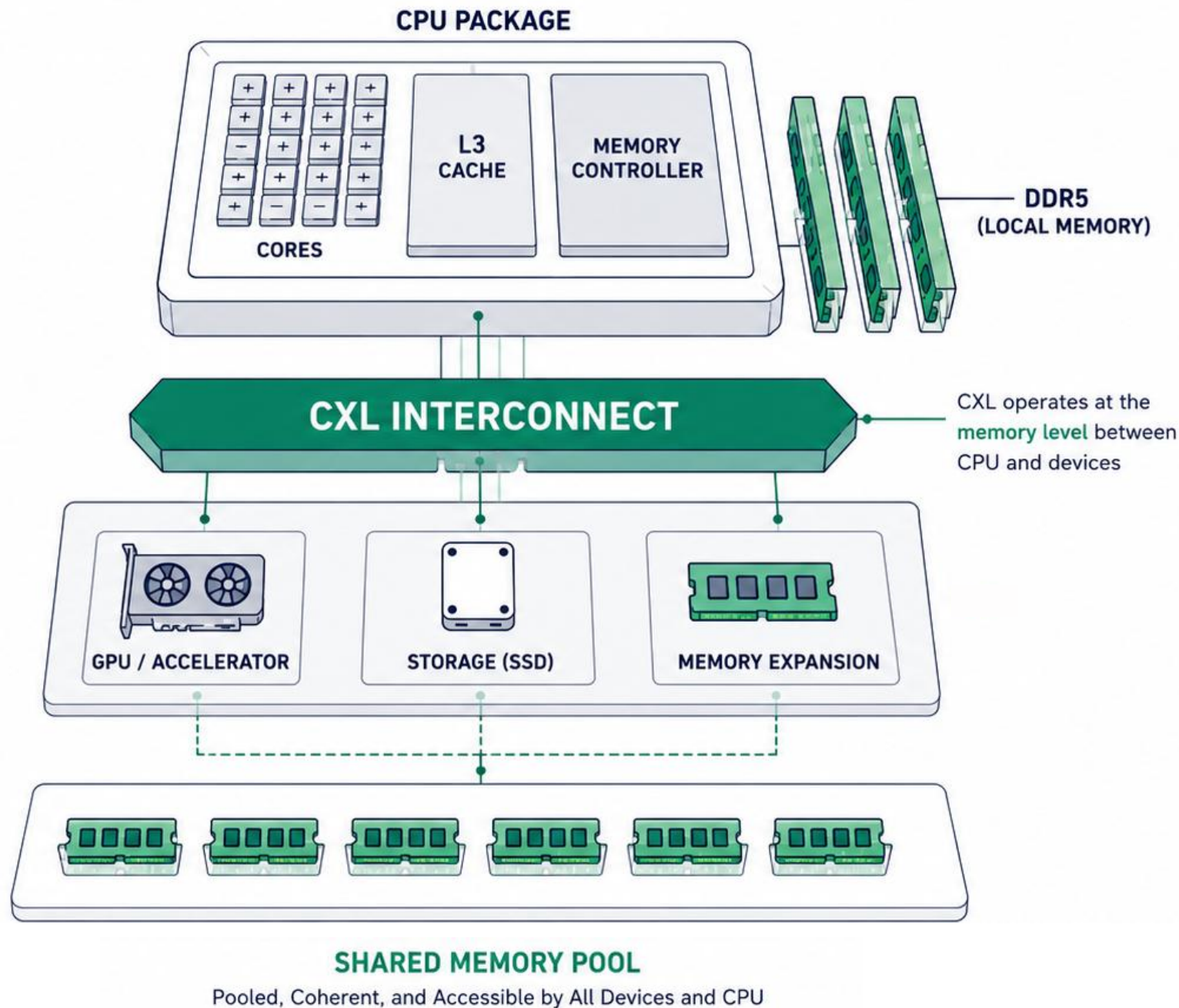
MEMORY WALL

Traditional architectures hit limits in memory capacity, bandwidth, and scalability, causing underutilization.



PCIe LIMITATIONS

PCIe was built for I/O, not memory sharing. It lacks cache coherency and efficient memory semantics.



Problem Statement

01



HIGH HARDWARE COST & AVAILABILITY

Enterprise CXL switches, Type-3 memory devices, and supporting platforms are expensive and not easily accessible for research, education, and early development.

02



LIMITED FLEXIBILITY OF STANDARD FIRMWARE & KERNEL ENVIRONMENTS

Stock OVMF firmware and Linux kernels lack the flexibility required for low-level CXL development and validation.

03



RAS VALIDATION IS DIFFICULT

Testing requires injecting hardware errors repeatedly. Real hardware fault injection is difficult, risky, and often requires specialized equipment.

04

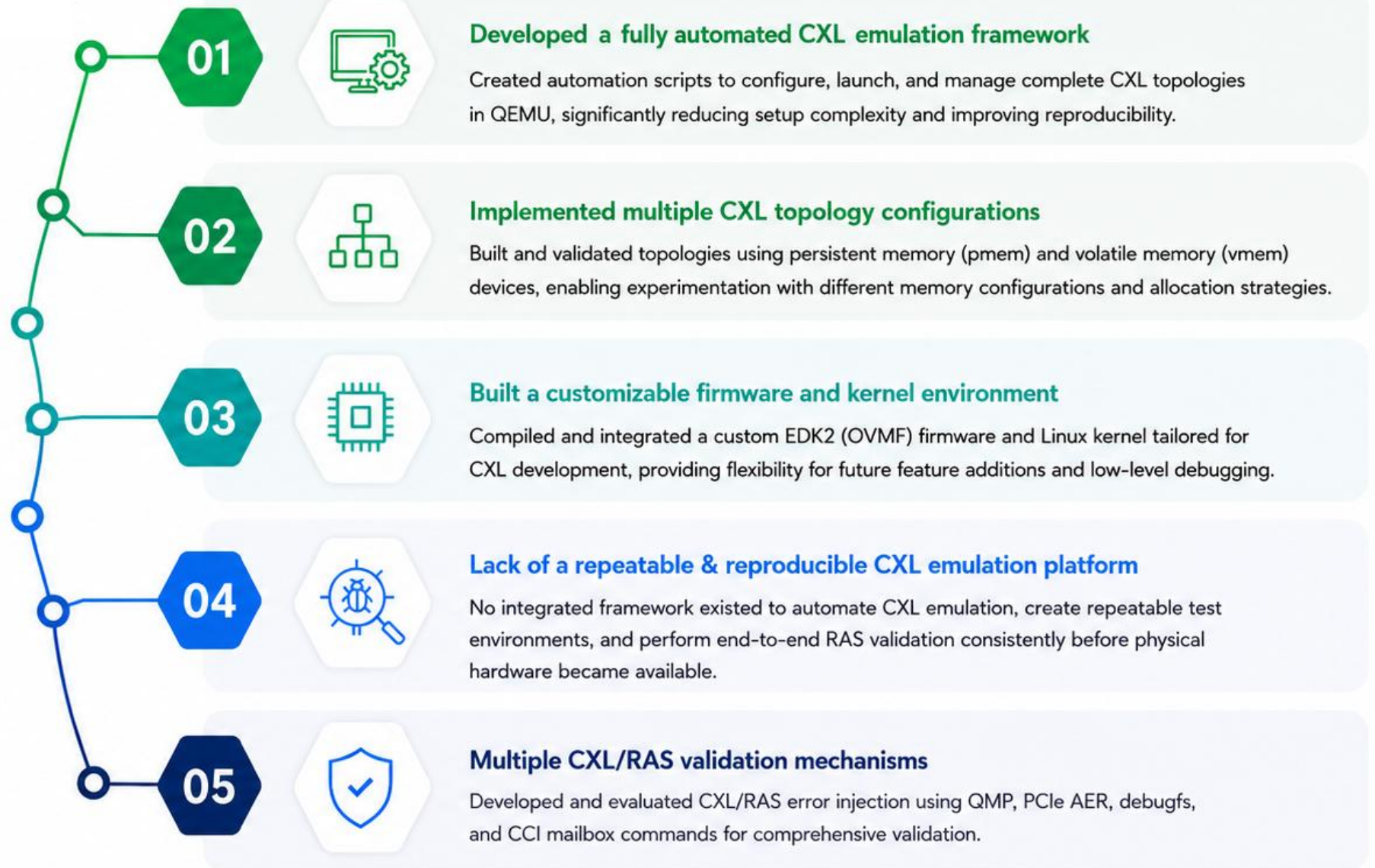


DIFFICULTY IN ESTABLISHING A CXL EMULATION ENVIRONMENT

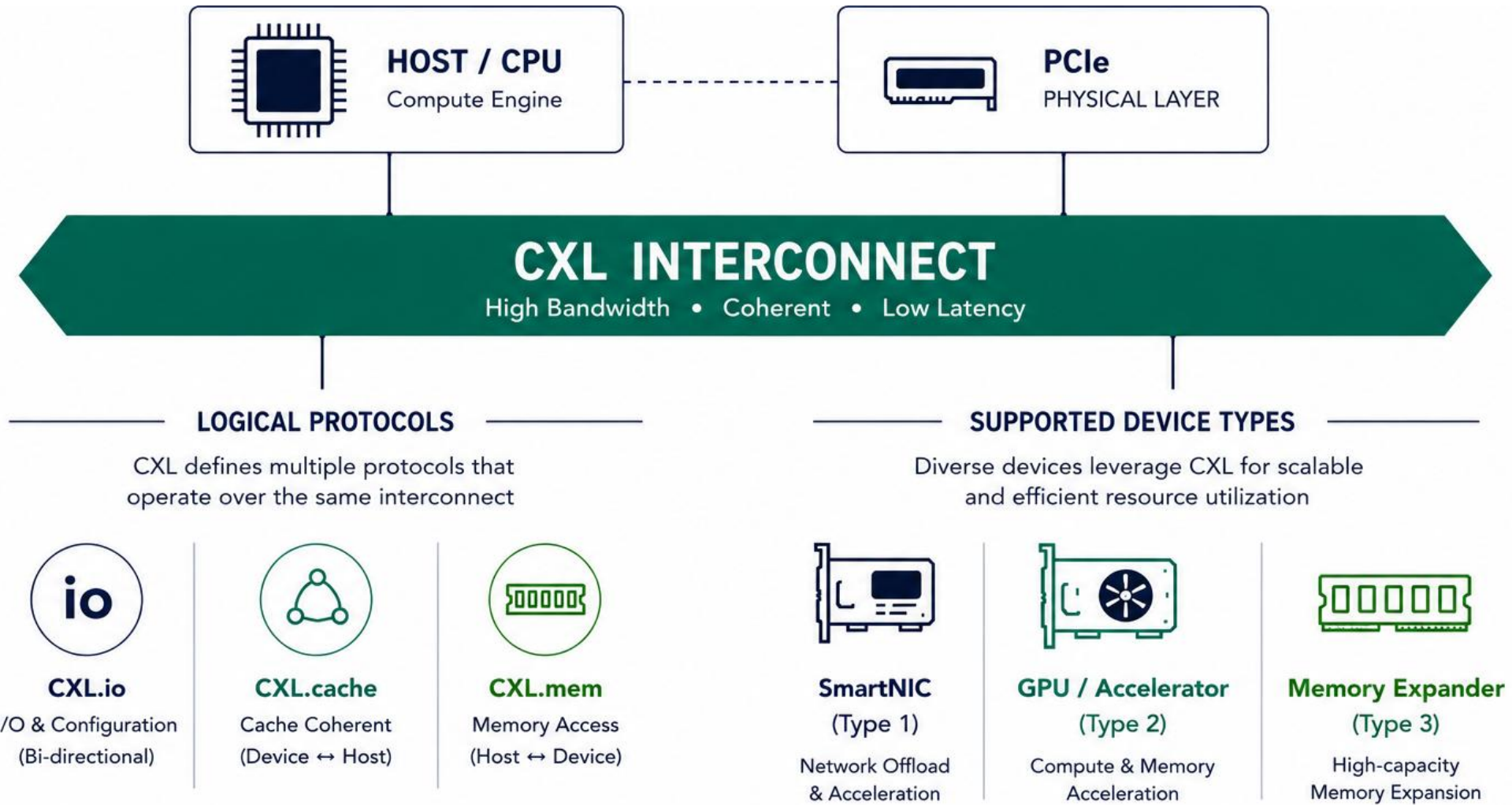
Deploying a functional CXL emulation environment required extensive manual configuration, increasing setup time and reducing reproducibility.



Proposed Solution



CXL (COMPUTE EXPRESS LINK)

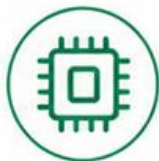


QEMU



FULL SYSTEM EMULATION

Emulates complete systems with CPU, PCIe, and CXL devices.



CXL DEVICE MODELING

Models CXL components including Type-3 memory and decoders.



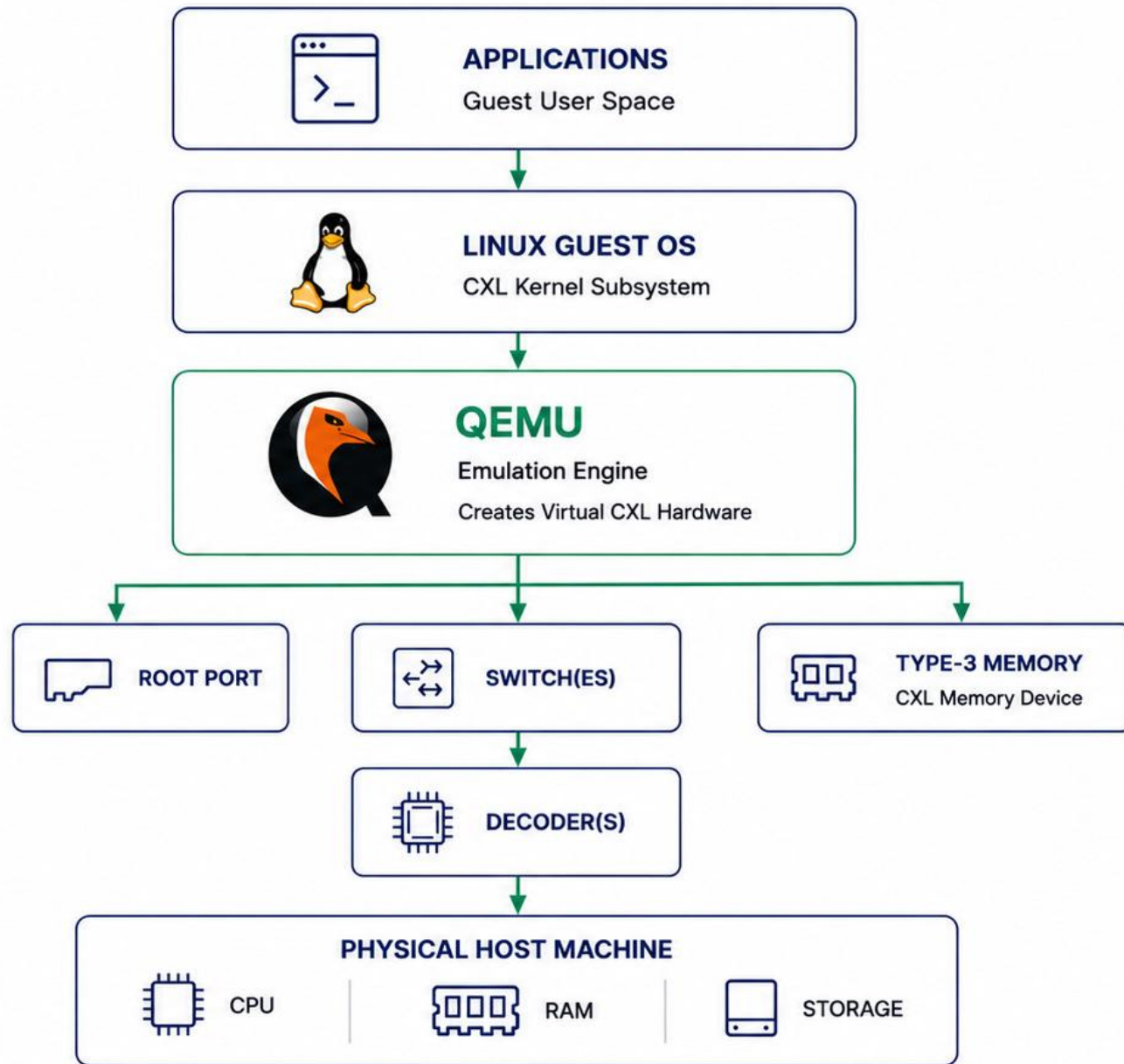
RAS ERROR INJECTION

Injects and validates AER and CXL RAS errors

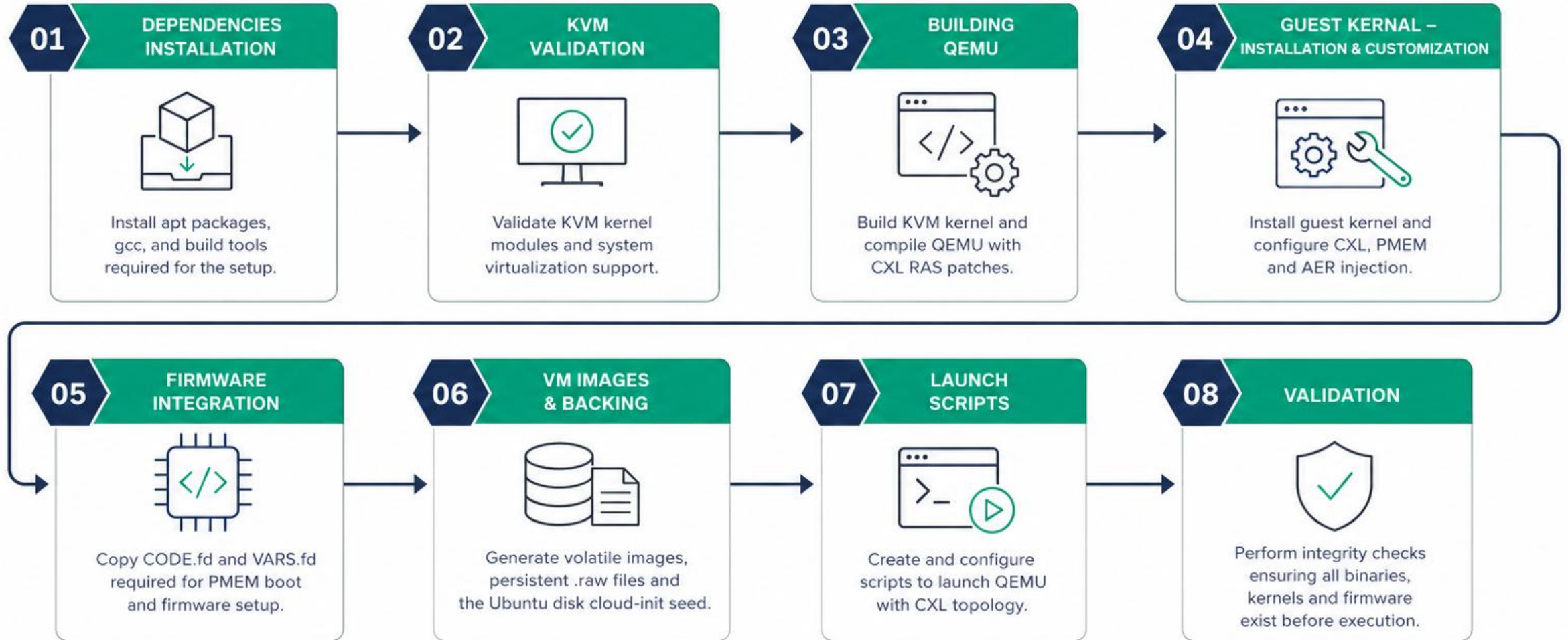


FLEXIBLE & REPEATABLE

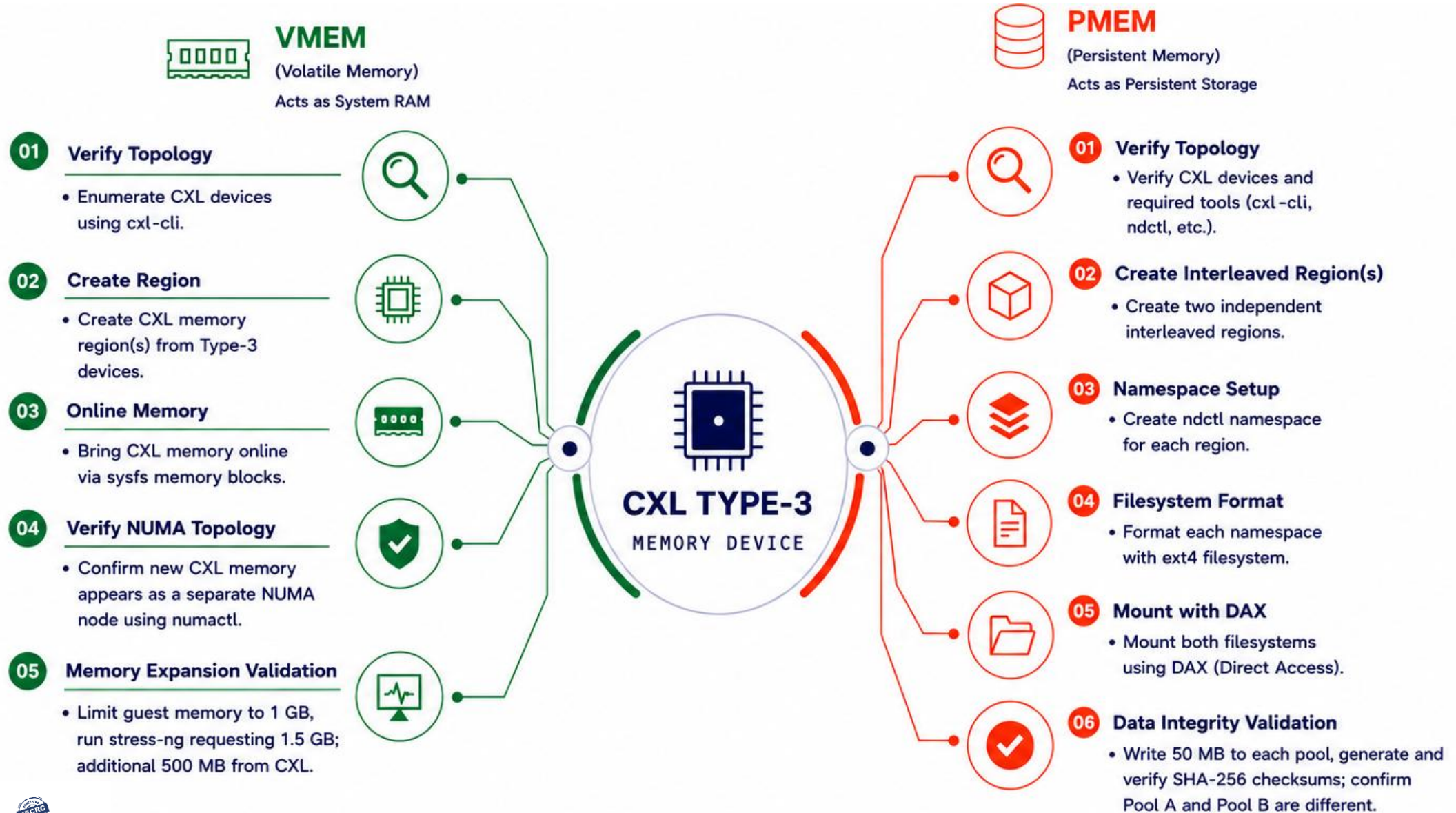
Create different topologies and failure scenarios for reliable testing.



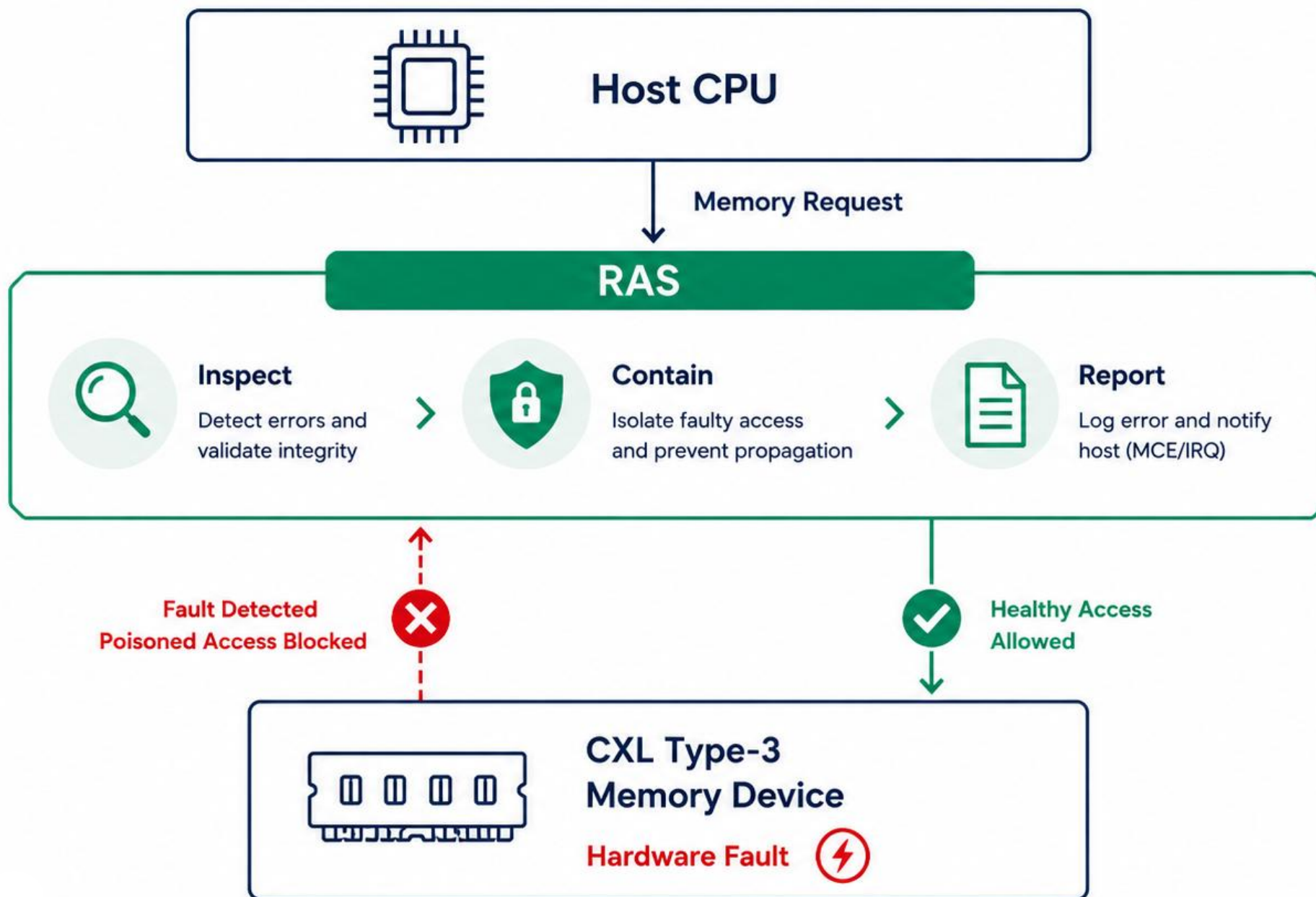
Environment Setup



CXL Memory Configuration



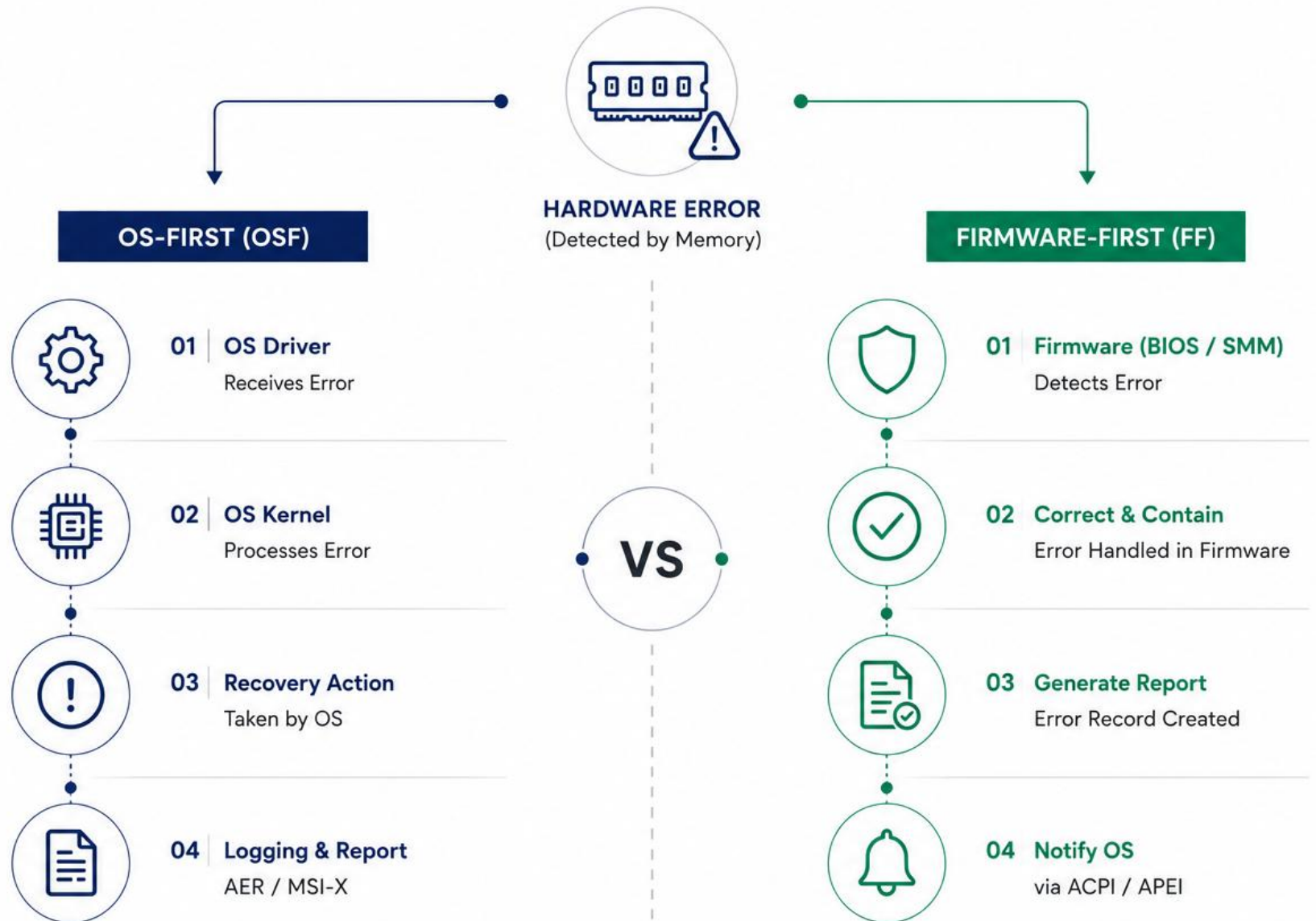
RAS



Types of Errors

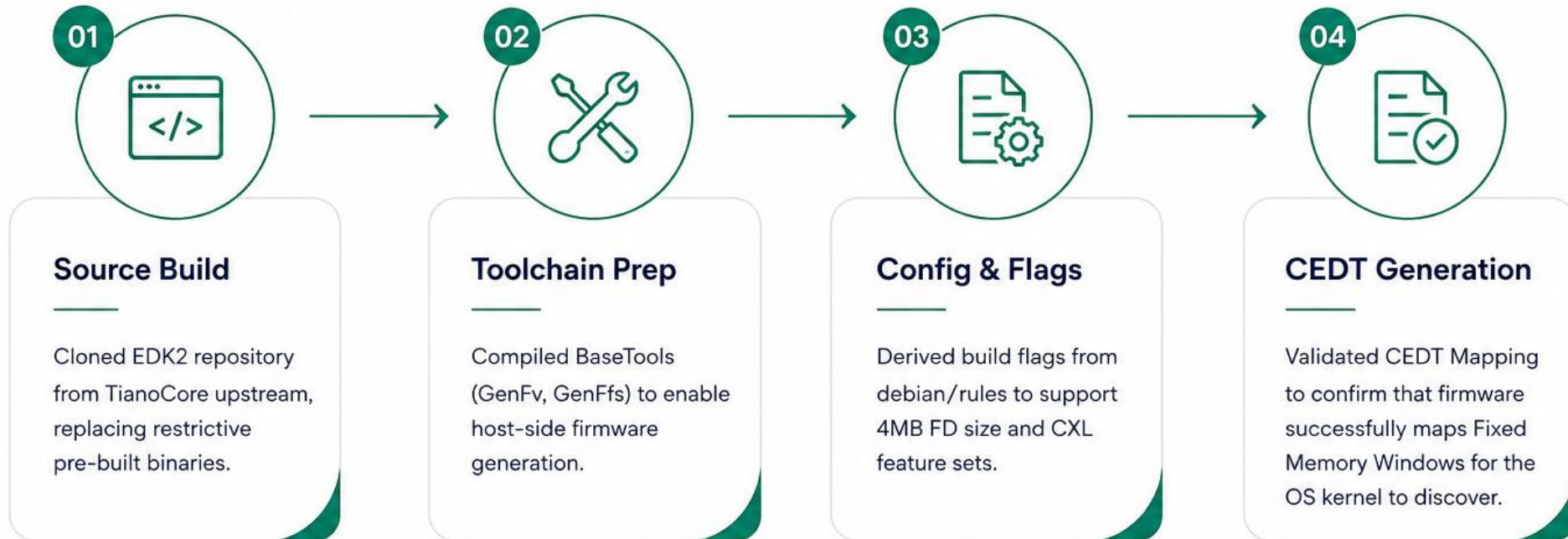


Error Handling Techniques



EDK2 Emulation

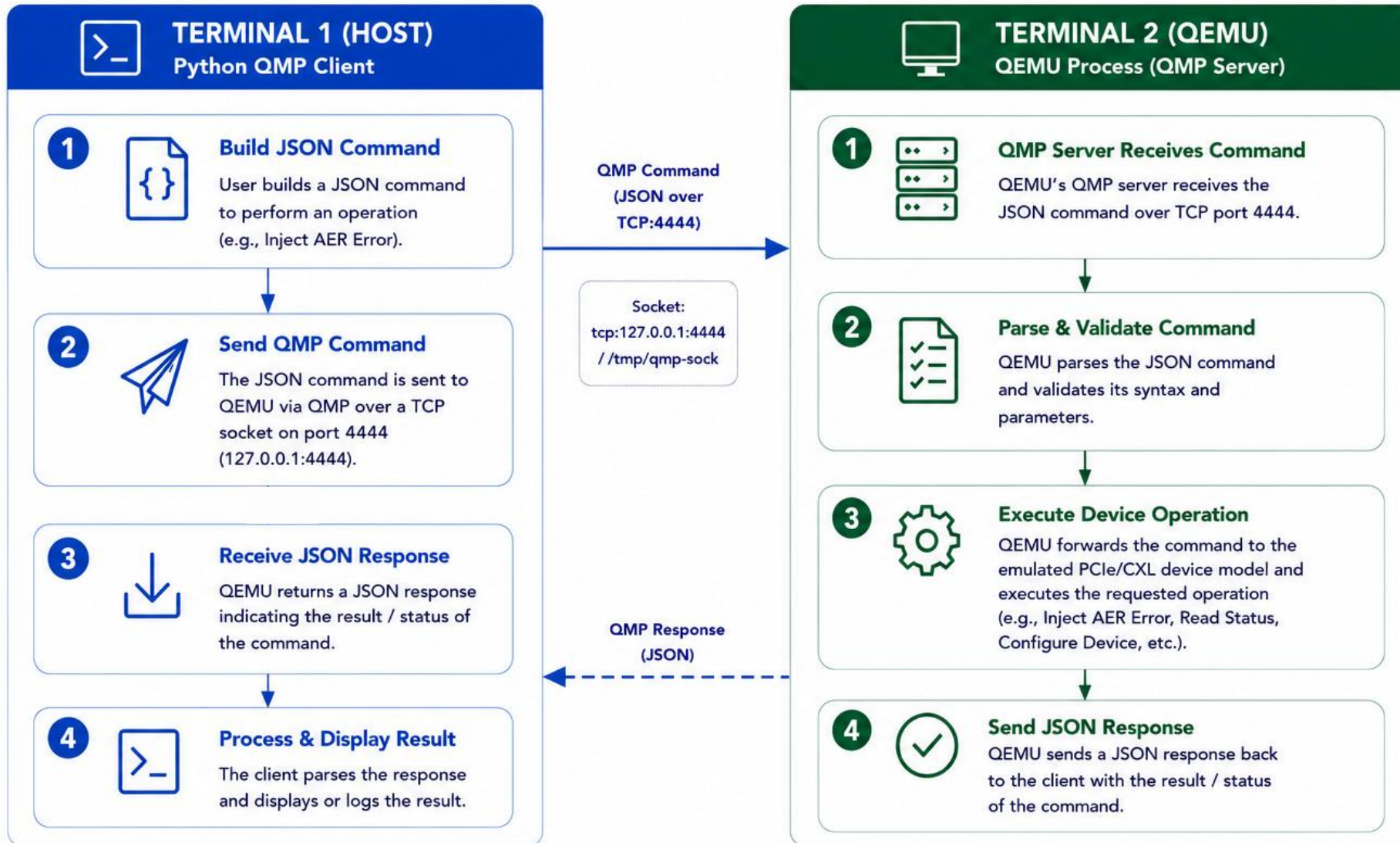
From Distro Binaries to Source: While standard Ubuntu OVMF packages emulate CXL hardware, they act as a RAS Black Box during development.



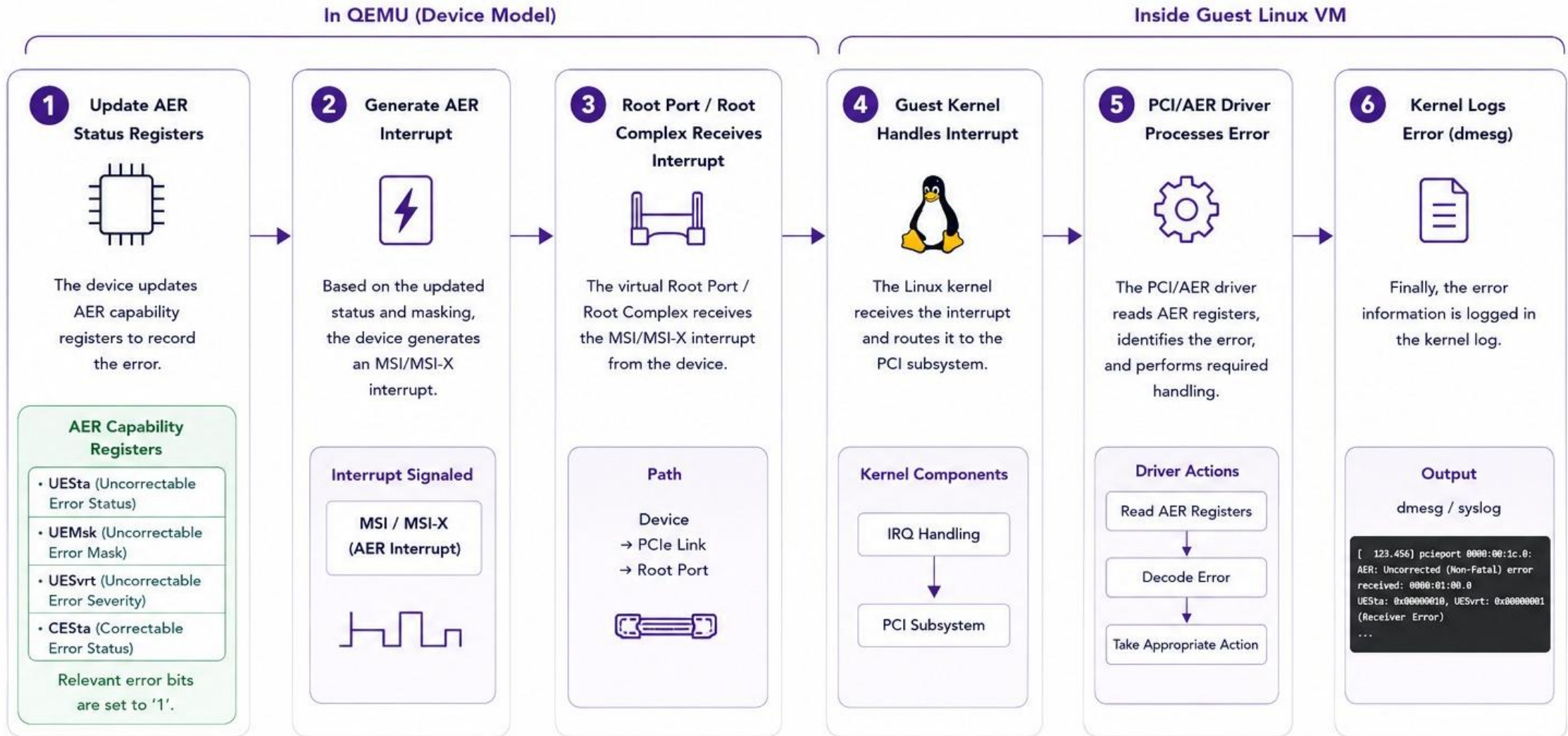
Impact of Custom OVMF

Unlike SeaBIOS, the EDK2-built OVMF generates the full ACPI hierarchy (`_HID`, `_CRS`) and Memory Map entries required for the OS to discover and hotplug CXL Type 3 devices.

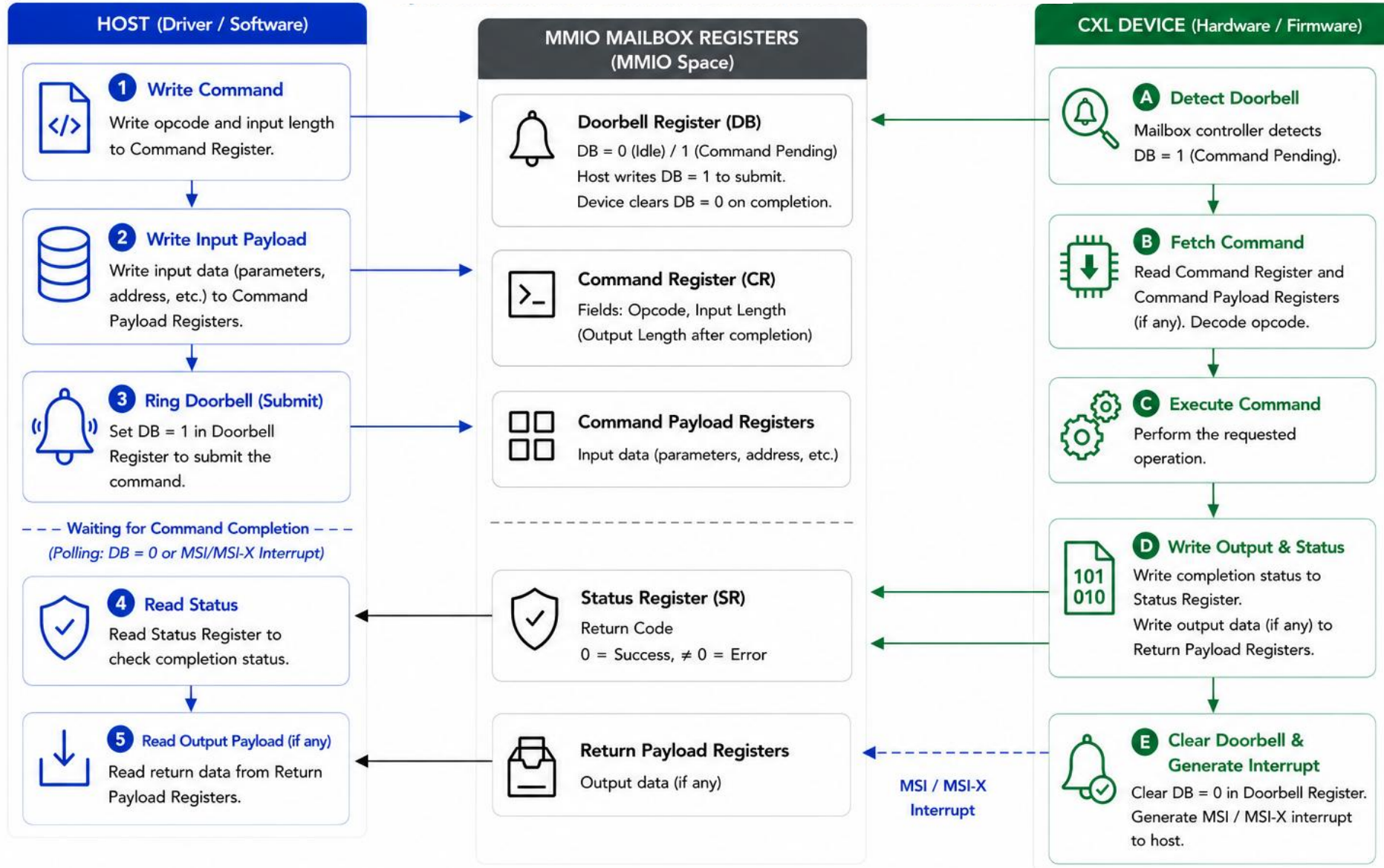
CXL RAS Error Injection Workflow via QMP



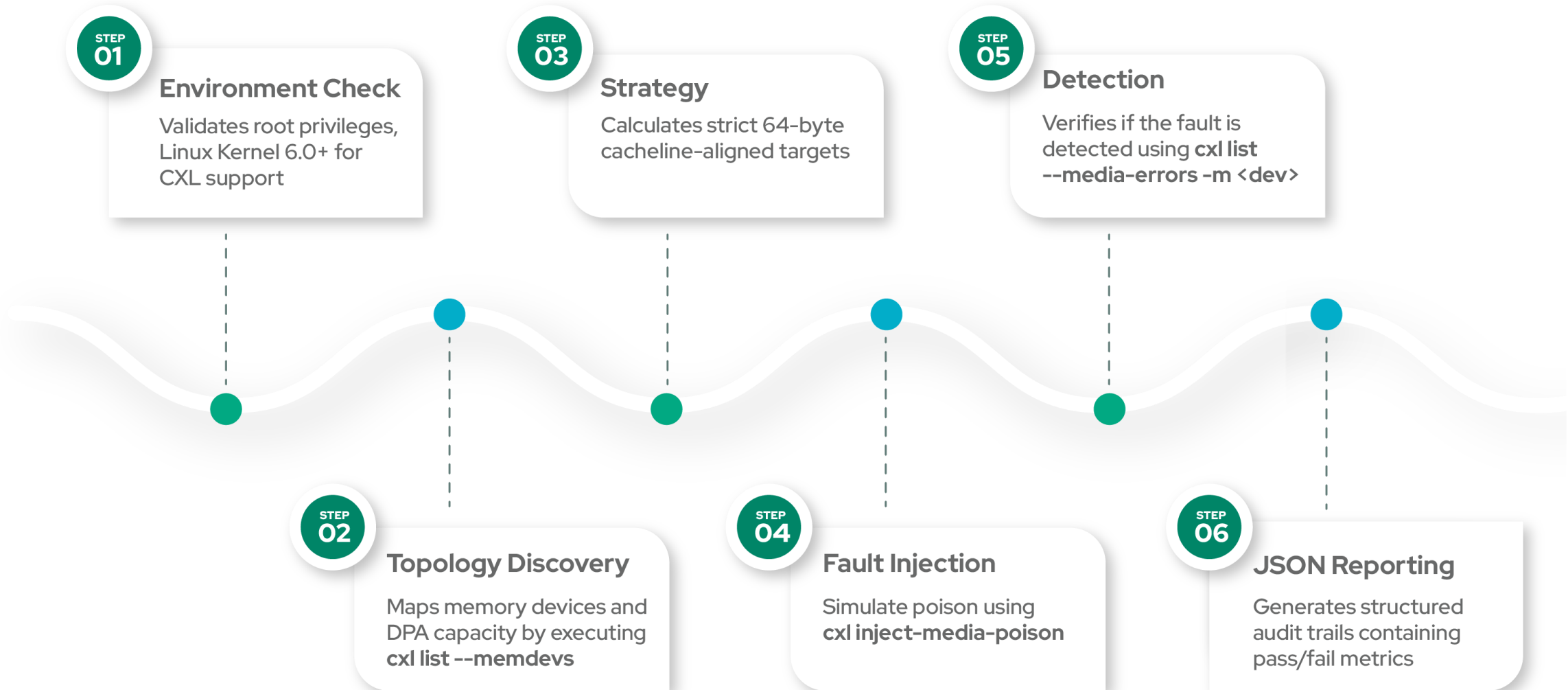
CXL AER Workflow



CXL CCI Workflow



Memory Poison Automation Pipeline



| Key Challenges

- **Evolving Specs & Toolchain Volatility:** Aligning the independently evolving CXL spec, QEMU models, and Linux subsystems was complex; minor version mismatches silently broke emulation.
- **Mapping Emulation Scope vs. Physical Silicon:** Determining the exact boundaries of RAS validation to identify which errors could be simulated in QEMU versus physical hardware.
- **Emulation Has a Ceiling:** Certain physical fault classes (e.g., thermal degradation) and specific Type-1 protocol errors cannot be modeled through QEMU or CCI.
- **Limited understanding of end-to-end hardware error propagation:** The propagation of CXL hardware errors across firmware, kernel, and applications was not well understood under realistic workloads.

| Key Learnings

- **Architecting Beyond PCIe Limitations:** Understood how the CXL Flex Bus resolves the "Memory Wall" via multiplexed cache coherency and dynamic pooling.
- **Software-Defined Validation:** Grasped the importance of QEMU to unblock early driver development and RAS validation without expensive pre-silicon hardware.
- **Firmware-OS Interdependencies & Enterprise RAS:** Understood UEFI's role in device enablement (ACPI/CDAT/CEDT) and why hyperscalers mandate Firmware-First (FF) fault containment.
- **Low-Level Component Communication (CCI):** Gained practical experience with MMIO mailboxes for dynamic configuration (e.g., Set Partition Info) and error record retrieval.
- **RAS as a Pillar for Evolving Technologies:** Realized early embedding of error detection and containment is essential to stabilize maturing architectures like CXL.

Conclusion

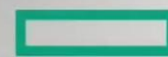
- **Automated Emulation & RAS Environment:** Scripted instant provisioning of QEMU-based CXL topologies, eliminating manual configuration for VMEM and RAS testing.
- **Validated Core CXL Functionality:** Emulated Type-3 (VMEM/PMEM) topologies to demonstrate memory expansion, coherent switching, and active spillover.
- **RAS Testing & Validation:** Proved hardware error lifecycle (detection to recovery) across realistic topologies via CCI, QMP, and AER injection.
- **Custom Software Stack Integration:** Bridged firmware and kernel spaces with a custom Linux kernel, EDK2/OVMF for Firmware-First architectures, and patched QEMU.
- **Automated Fault Injection Pipeline:** Engineered a Python framework for dynamic topology discovery, strategic DPA poisoning, and automated JSON reporting.

Future Scope

- **DCD & Fabric Manager Support:** Extend beyond a single host to enable elastic, on-demand pooled capacity across a fabric.
- **Multi-Host Topologies:** Validate coherency and RAS behavior for memory shared across multiple independent hosts.
- **Advanced Firmware-First Integration:** Deepen custom UEFI development to mirror real-world enterprise error containment.
- **Multi-Emulator Exploration:** Evaluate cycle-accurate emulators to test configurations beyond QEMU's current scope.
- **Real-Hardware Correlation:** Execute the validation suite against physical CXL silicon to confirm emulated behavior.

CXL Enablement

with QEMU-Based Emulation

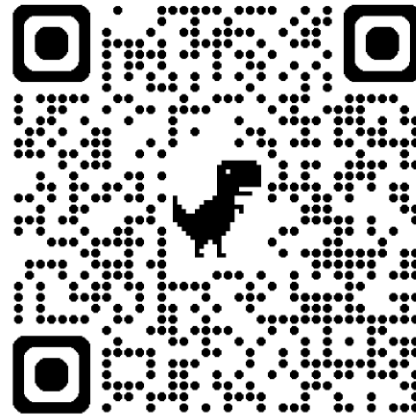


Hewlett Packard
Enterprise



We've made the complete source code and documentation for the project publicly available on our Github.

github.com/CXL-Enablement-with-QEMU-Based-Emulation



Thank You

